30-Day Backend Mastery Guide (Detailed Daily Instructions)

This guide is designed to teach backend development step-by-step using Node.js, Express, TypeScript, and MongoDB. Each day includes: - What you are building - What you must understand - What to improve if you have extra time This is not theory. This is practical engineering training.

Stack Overview

Core Industry Stack: Node.js, Express.js, TypeScript, MongoDB, Mongoose, JWT Authentication, bcrypt, dotenv, Postman, Git/GitHub. Upgrade Stack (later stage): PostgreSQL, Prisma ORM, Redis, Docker, Swagger, CI/CD pipelines.

Daily Roadmap

Day 1
Build your first Express + TypeScript server. Understand project structure (app.ts, server.ts, routes). Create /health API. Upgrade: use .env for PORT.

Day 2
Learn routing system (GET, POST, PUT, DELETE). Build a Notes API in memory. Understand request-response cycle deeply. Upgrade: split routes into folders.

Day 3
Middleware concept: functions between request and response. Build request logger middleware. Upgrade: create custom auth-check middleware.

Day 4
Error handling system. Learn try/catch and global error middleware. Build safe calculator API. Upgrade: custom error class.

Day 5
REST API design principles. Learn how real APIs are structured. Build Task API v1. Upgrade: clean endpoint naming conventions.

Day 6
Query parameters and filtering. Build product filtering API. Upgrade: add sorting + pagination.

Day 7
Mini project: Task Manager API (no database). Focus on structure and logic. Upgrade: refactor into MVC pattern.

Day 8
Connect MongoDB. Learn database connection and schema basics. Build user save API. Upgrade: validation rules in schema.

Day 9
CRUD operations with MongoDB. Build Notes API with database. Upgrade: service layer separation.

Day 10
Schema design thinking. Learn how data is structured in real apps. Build Todo system v2. Upgrade: better relationships.

Day 11
User-task relationship modeling. Understand one-to-many relationships. Upgrade: nested population concept.

Day 12
Pagination system. Learn skip/limit logic. Upgrade: infinite scroll backend design.

Day 13
Search system. Build text search API. Upgrade: indexing concept.

Day 14
Full review project: Task Manager with database + structure cleanup.

Day 15
Password hashing using bcrypt. Build secure signup API. Upgrade: password strength validation.

Day 16
JWT authentication. Build login system. Understand token generation. Upgrade: token expiration logic.

Day 17
Protected routes middleware. Build private API endpoints. Upgrade: role-based access concept.

Day 18
Role system (admin/user). Learn authorization vs authentication. Upgrade: permission matrix.

Day 19
Token handling and refresh concept. Upgrade: secure token storage idea.

Day 20
Input validation using Zod/Joi. Build validated signup system. Upgrade: centralized validation layer.

Day 21
Complete authentication system project. Signup, login, protected routes.

Day 22
File upload system using multer. Build image upload API. Upgrade: file type validation.

Day 23
Environment variables deep usage. Secure configuration setup.

Day 24
Rate limiting concept. Prevent abuse of APIs. Upgrade: IP-based throttling.

Day 25
Caching basics. Learn performance improvement. Upgrade: Redis introduction concept.

Day 26
Professional backend folder structure (controllers, services, routes).

Day 27
Start Blog API (real-world backend project). Design models first.

Day 28
Finish Blog API (CRUD + structure + validation).

Day 29
Add authentication + roles to Blog API.

Day 30
Deploy project using Render/Railway. Push to GitHub portfolio. Final cleanup.

Final Warning: If you only copy code, you will stay beginner. If you understand structure, you become engineer. Always ask: Why does this exist? Where is this used in real companies? How would I scale this system?